

Instruction Selection & Scheduling via SMT

CS517

Raine Wheary, Christine Lin
[GitHub project](#)

We consider the problem of backend code generation in a compiler. By this point in the pipeline, we can understand the input program as having been reduced to a graph of basic operations in some intermediate representation (IR). The backend must further lower these operations into machine instructions.

For linear control flow (i.e. within a single basic block), there are three major tasks to be solved:

- **Instruction selection:** Modern CPU architectures often have complex macroinstructions encoding more than one basic operation. For example, the x86-64 instruction `mov rax, [rcx + 4*rdx + 28]` involves a left shift, two additions, and a load. Other examples include stack operations, multiple loads and stores (e.g. ARM's `ldp` and `stp`), and fused-multiply-add. A compiler will prefer to emit these instructions when applicable, but it can be a net loss if it means intermediate results are no longer available and need to be recomputed. In the last example, if the address value `rcx + 4*rdx + 28` was used in many places, a compiler should compute this effective address with `lea` and reuse it, rather than doing the calculation inside the `mov`.
- **Instruction scheduling:** Even when two machine instructions could be independent of each other, their order relative to each other can impact the performance of the generated code. We model a pipelined but not out-of-order processor, where a pipeline stall occurs if an instruction's inputs are not available.
- **Register allocation:** Programs in IR form use an unlimited number of virtual registers, but these need to be mapped to a limited number of physical registers on the machine.

Each of these tasks should be understood as a search problem. Classically, the backend is split into three phases, with instruction selection, instruction scheduling, and register allocation being performed in order [1]. However, this architecture has its limitations. When deciding whether to emit a higher-latency macroinstruction, there are situations where the optimal choice depends on register pressure or on the ability of the scheduler to avoid a pipeline stall.

Register allocation is well known to be NP-complete, and is typically solved with heuristic algorithms in practice. We will focus on an integrated algorithm for first two problems via reduction to SMT.

Problem statement

We will adopt the convention of coloring words and variables related to the IR (the input) blue and those related to the machine program (the output) red.

We assume there is a fixed set of **IR opcodes** C and each opcode $\alpha \in C$ has an associated **arity**.

As input, we start with an **IR program**, which is a sequence P of **IR instructions**. Each **IR instruction** consists of an opcode and a tuple of arguments matching the opcode's arity.

$$P := \{P_1, \dots, P_N\}$$
$$P_i ::= \alpha(t_1, \dots, t_m), \quad \alpha \in C, \quad m = \text{arity}(\alpha), \quad 1 \leq t_1, \dots, t_m < i$$

The requirement $1 \leq t_1, \dots, t_m < i$ captures the idea that the input program is a DAG, because each parameter of an **IR instruction** is the index of some previous instruction. This also means that the first instruction must be nullary.

In addition, a subset of program indices $R \subseteq \{1, \dots, N\}$ are designated as **results**. An index $i \in R$ being contained in the set means that the result of the instruction P_i must be materialized in the **machine program**, i.e. it cannot be coalesced into an intermediate computation of a **machine instruction**.

References

- [1] G. S. H. Blindell, "Survey on Instruction Selection: An Extensive and Modern Literature Review." [Online]. Available: <https://arxiv.org/abs/1306.4898>